Introduction to the Theory of Computation

Lecture 13. Undecidable Langauges

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

Most Languages Are Undecidable

Theorem 1
Almost all languages are undecidable.

Proof.

- Each TM is a finite string $\Rightarrow$ countably many TMs.
- Each TM decides at most one language $\Rightarrow$ countably many decidable languages.
- $|\mathcal{P}(\{0,1\}^*)| = 2^{\aleph_0}$ is uncountable (Cantor).

□

Halting Encodes Mathematics

```
from itertools import count, product

for s in count(3):
    R = range(1, s+1)
    for a, b, c, n in product(R, repeat=4):
        if n >= 3 and a**n + b**n == c**n:
            exit()
```

This program halts $\iff$ Fermat's Last Theorem is **false**.

Halting Encodes Mathematics

```
from itertools import count

def prime(k):
    return k > 1 and all(k % d for d in range(2, k))

for n in count(4, 2):
    if not any(prime(p) and prime(n-p)
               for p in range(2, n)):
        exit()
```

This program halts $\iff$ Goldbach's Conjecture is **false**.

The Halting Problem

Definition 2

$$L_{\text{halt}} = \{(\alpha, x) : M_\alpha \text{ halts on input } x\}.$$

In words: given a program and an input, does the program halt?

Theorem 3 (Turing 1936)
L_{halt} is undecidable.

Idea: Diagonalization

Rows: all TMs $M_0, M_1, \ldots$ Columns: all strings $s_0, s_1, \ldots$

Entry (i, j): what M_i does on input s_j.

	s_0	s_1	s_2	s_3	$\cdots$
M_0	R	A	R	L	$\cdots$
M_1	R	L	A	R	$\cdots$
M_2	L	A	R	L	$\cdots$
M_3	A	R	L	A	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

A = accept

R = reject

L = loop forever

Idea: Diagonalization

Rows: all TMs $M_0, M_1, \ldots$ Columns: all strings $s_0, s_1, \ldots$

Entry (i, j): what M_i does on input s_j.

	s_0	s_1	s_2	s_3	$\cdots$
M_0	R	A	R	L	$\cdots$
M_1	R	L	A	R	$\cdots$
M_2	L	A	R	L	$\cdots$
M_3	A	R	L	A	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
L_{flip}	A	A	A	R	$\cdots$

A = accept

R = reject

L = loop forever

Flip diagonal:

A $\rightarrow$ R

R $\rightarrow$ A

L $\rightarrow$ A

Idea: Diagonalization

Rows: all TMs $M_0, M_1, \ldots$ Columns: all strings $s_0, s_1, \ldots$

Entry (i, j): what M_i does on input s_j.

	s_0	s_1	s_2	s_3	$\cdots$
M_0	R	A	R	L	$\cdots$
M_1	R	L	A	R	$\cdots$
M_2	L	A	R	L	$\cdots$
M_3	A	R	L	A	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
L_{flip}	A	A	A	R	$\cdots$

A = accept

R = reject

L = loop forever

Flip diagonal:

A $\rightarrow$ R

R $\rightarrow$ A

L $\rightarrow$ A

L_{flip} differs from every M_i on input $s_i \Rightarrow L_{\text{flip}}$ is **not** any row.

Definition of L_{flip}

Definition 4

$$L_{\text{flip}} = \{\alpha : M_\alpha \text{ does not accept } \alpha\}.$$

That is, $\alpha \in L_{\text{flip}} \iff M_\alpha$ rejects α or loops on α.

Lemma 5

L_{flip} is undecidable.

L_{flip} is Undecidable

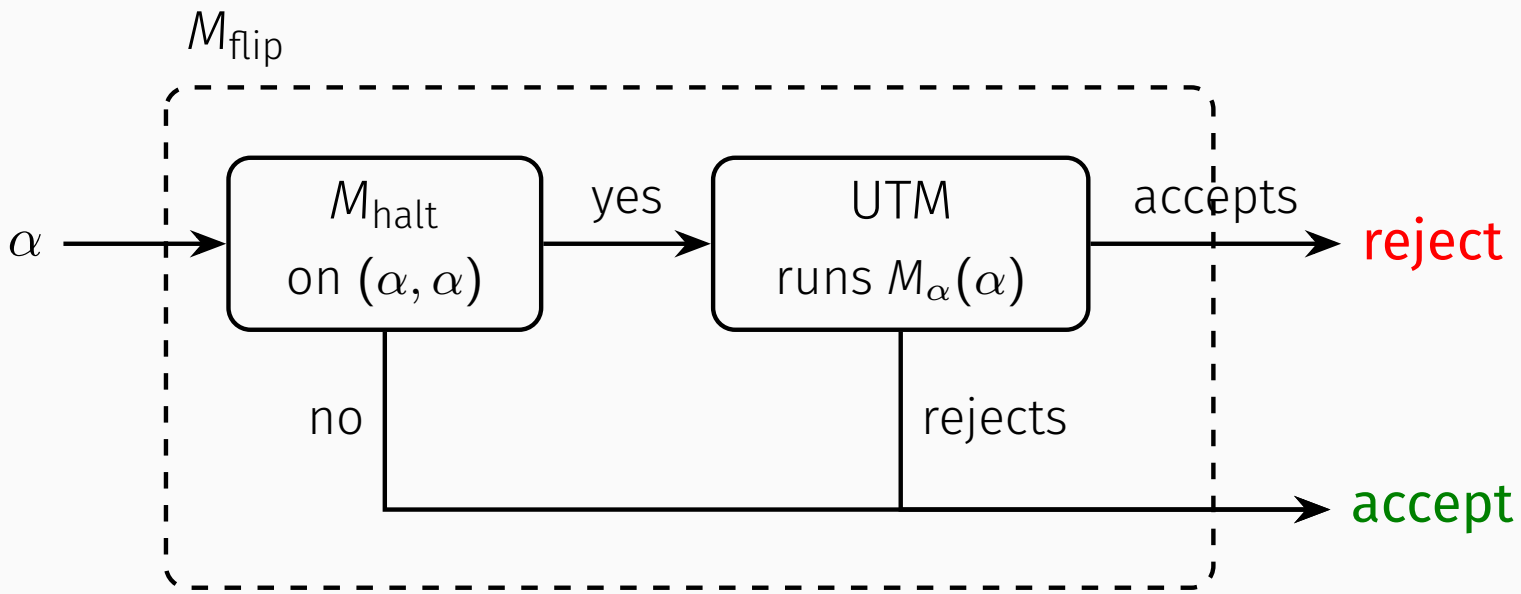
Proof.
Suppose M_β decides L_{flip}, so $L(M_\beta) = L_{\text{flip}}$.

Case 1: $\beta \in L_{\text{flip}}$. By definition, M_β does not accept β, so $\beta \notin L(M_\beta) = L_{\text{flip}}$.

Case 2: $\beta \notin L_{\text{flip}}$. By definition, M_β accepts β, so $\beta \in L(M_\beta) = L_{\text{flip}}$.

Both cases are contradictions. □

L_{halt} is Undecidable: The Idea



If M_{halt} exists, then M_{flip} decides L_{flip}. Impossible!

L_{halt} is Undecidable

Proof.
Suppose TM M_{halt} decides L_{halt}. Build M_{flip}: on input α, run M_{halt} on (α, α).

- If it rejects (M_α loops on α): accept.
- If it accepts: simulate M_α on α with a UTM, and output the **opposite**.

M_{flip} always halts and decides L_{flip}, contradicting the lemma. □

Remarks on the Halting Problem

- Undecidable **in general**, but decidable for restricted classes (e.g. space-bounded TMs).
- Halting of specific short programs encodes famous open conjectures (Goldbach, Riemann Hypothesis).

L_{accept} is Undecidable

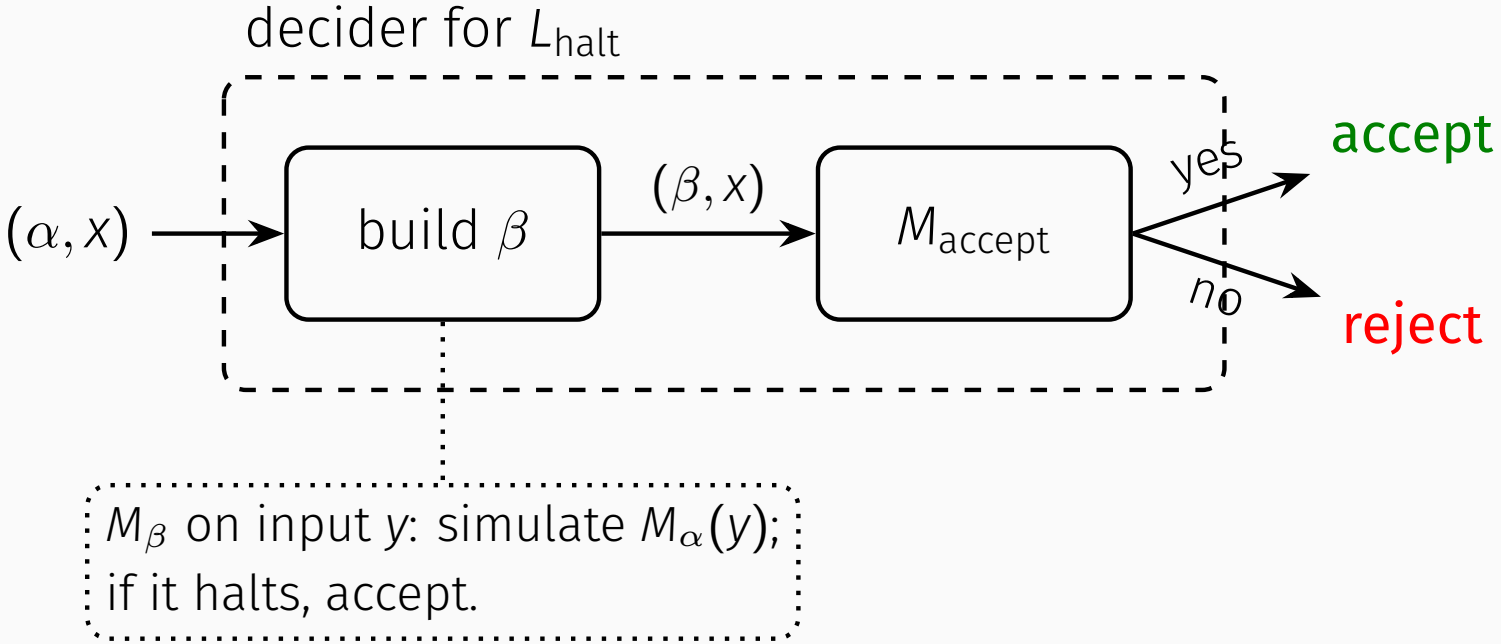
Definition 6

$$L_{\text{accept}} = \{(\alpha, x) : M_\alpha \text{ accepts } x\}.$$

Lemma 7

L_{accept} is undecidable.

L_{accept} is Undecidable: The Idea



$$M_\alpha \text{ halts on } x \iff M_\beta \text{ accepts } x.$$

L_{accept} is Undecidable

Proof.
Suppose, for contradiction, M_{accept} decides L_{accept}. We construct a new TM that decides L_{halt}.

Given (α, x), build M_β: simulate $M_\alpha(x)$, accept iff it halts.

Run M_{accept} on (β, x) — this decides $L_{\text{halt}}(\alpha, x)$.
Contradiction. □

L_{empty} is Undecidable

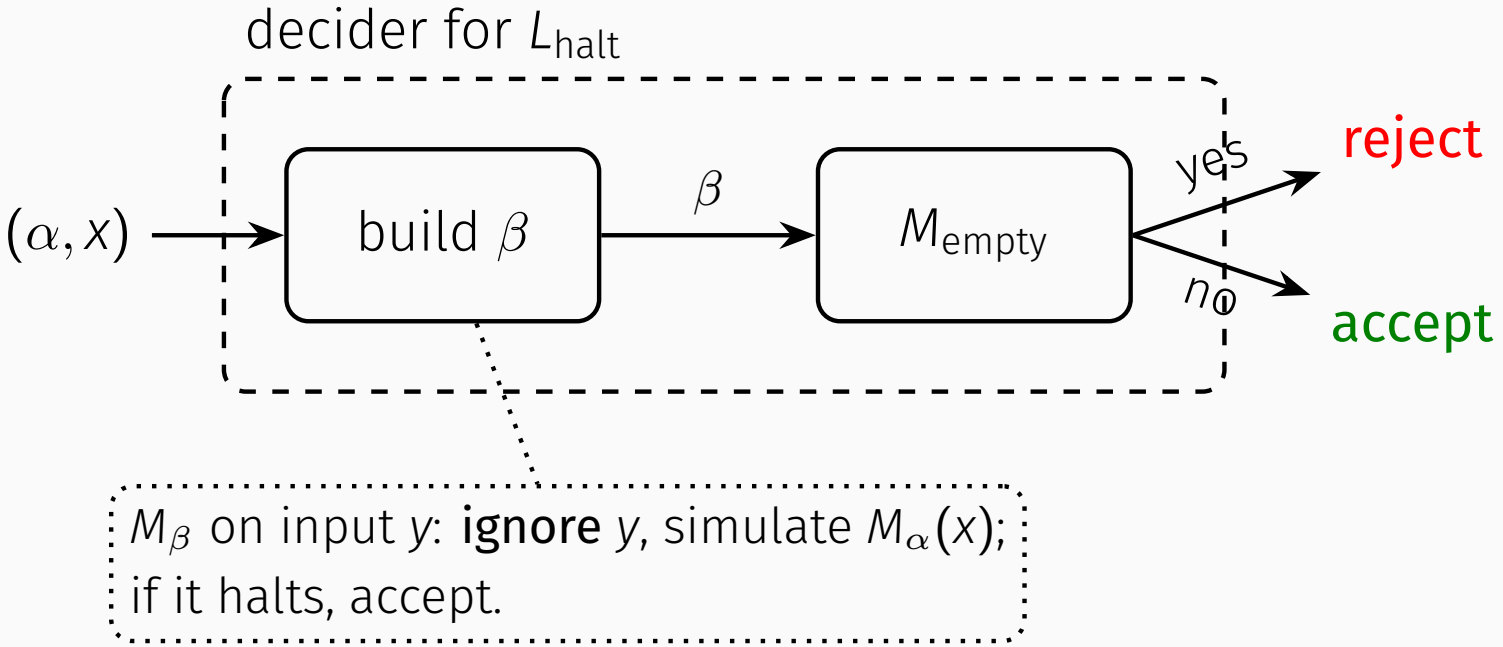
Definition 8

$$L_{\text{empty}} = \{\alpha : L(M_\alpha) = \emptyset\}.$$

That is, M_α accepts no input at all.

Lemma 9
L_{empty} is undecidable.

L_{empty} is Undecidable: The Idea



$$M_\alpha \text{ halts on } x \iff L(M_\beta) \neq \emptyset.$$

L_{empty} is Undecidable

Proof.
Suppose M_{empty} decides L_{empty}. We decide L_{halt} as follows.

Given (α, x), compute β, where M_β on input y ignores y, simulates $M_\alpha(x)$, and accepts if it halts. Then

$$L(M_\beta) = \begin{cases} \{0,1\}^* & M_\alpha \text{ halts on } x, \\ \emptyset & \text{otherwise.} \end{cases}$$

So $(\alpha, x) \in L_{\text{halt}} \iff \beta \notin L_{\text{empty}}$: run M_{empty} on β and flip the answer. Contradiction. $\square$

L_{regular} is Undecidable

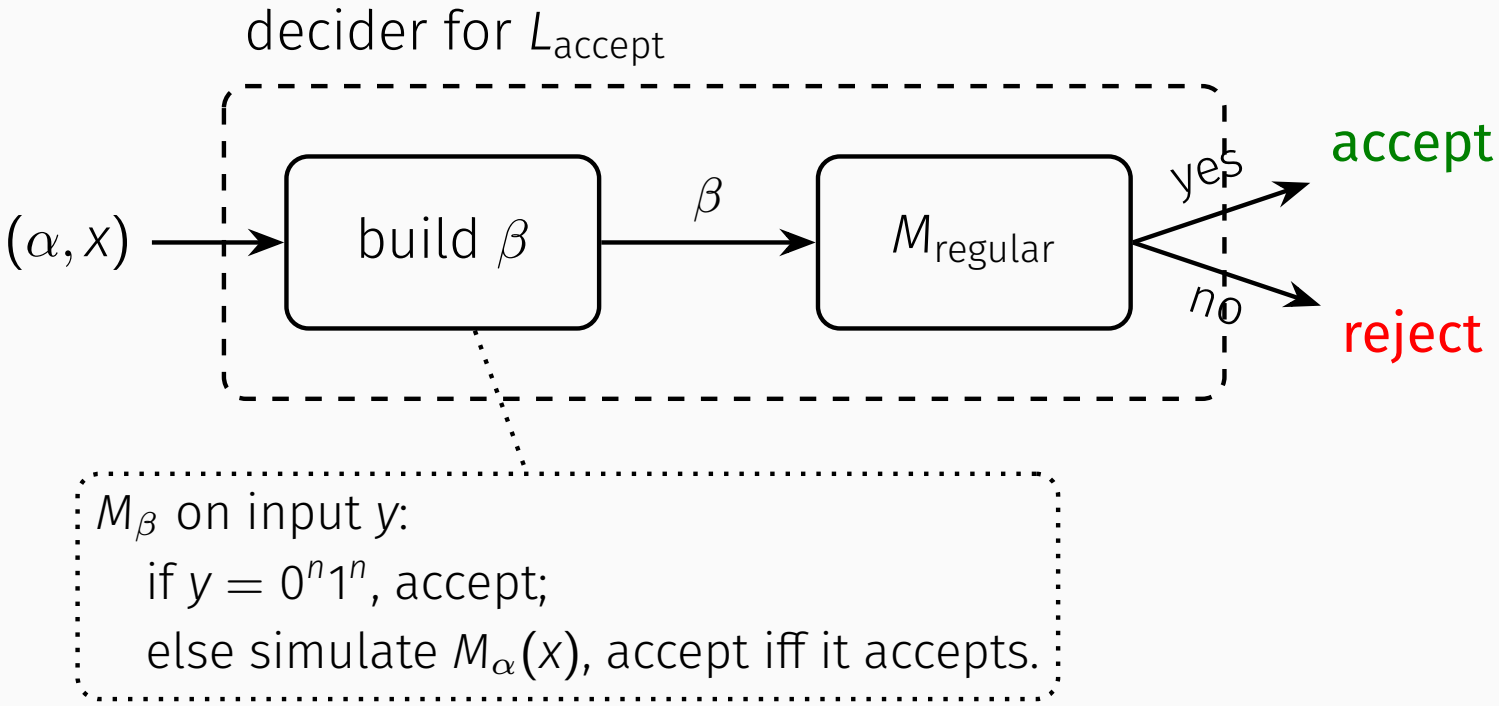
Definition 10

$$L_{\text{regular}} = \{\alpha : L(M_\alpha) \text{ is regular}\}.$$

Lemma 11

$L_{regular}$ is undecidable.

L_{regular} is Undecidable: The Idea



$$M_\alpha \text{ accepts } x \iff L(M_\beta) \text{ is regular.}$$

L_{regular} is Undecidable

Proof.
Suppose M_{regular} decides L_{regular}. We decide L_{accept} as follows.

Given (α, x), compute β, where M_β on input y: if $y = 0^n 1^n$, accept; otherwise simulate $M_\alpha(x)$ and accept iff it accepts. Then

$$L(M_\beta) = \begin{cases} \{0,1\}^* & M_\alpha \text{ accepts } x \quad \text{(regular)}, \\ \{0^n 1^n : n \geq 0\} & \text{otherwise} \quad \text{(not regular)}. \end{cases}$$

So $(\alpha, x) \in L_{\text{accept}} \iff \beta \in L_{\text{regular}}$: run M_{regular} on β.
Contradiction. $\square$

A Pattern

All these undecidable problems ask about a program's **behavior**, not its code:

- Does M_α halt on x?
- Does M_α accept x?
- Does M_α accept nothing?
- Is the set of strings M_α accepts regular?

Rice's theorem (preview). Every nontrivial property of $L(M_\alpha)$ is undecidable.

Virus Detection is Undecidable

Virus (Cohen): a program that, whenever run, writes a (possibly evolved) copy of itself elsewhere.

Theorem 12 (Cohen)
No TM decides whether a given program is a virus.

Proof idea. Given (α, x), build program P: simulate $M_\alpha(x)$ in an isolated area; if it halts, copy P elsewhere. Then

$$P \text{ is a virus} \iff M_\alpha \text{ halts on } x.$$

F. Cohen, *Computer Viruses: Theory and Experiments*, Computers & Security, 6(1):22–35, 1987.
F. Cohen, *Computational Aspects of Computer Viruses*, Computers & Security, 8(4):325–344, 1989.

L_{equal} is Undecidable

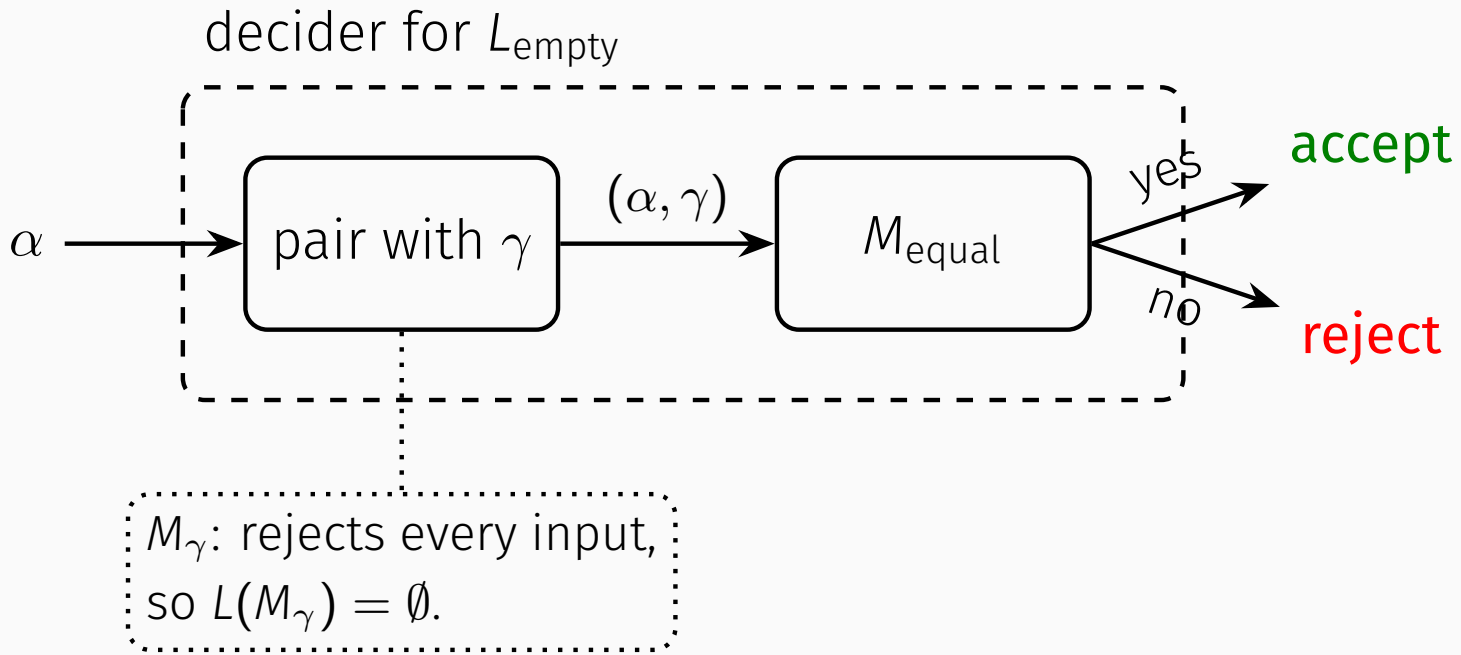
Definition 13

$$L_{\text{equal}} = \{(\alpha, \beta) : L(M_\alpha) = L(M_\beta)\}.$$

Lemma 14

L_{equal} is undecidable.

L_{equal} is Undecidable: The Idea



$$L(M_\alpha) = \emptyset \iff L(M_\alpha) = L(M_\gamma).$$

L_{equal} is Undecidable

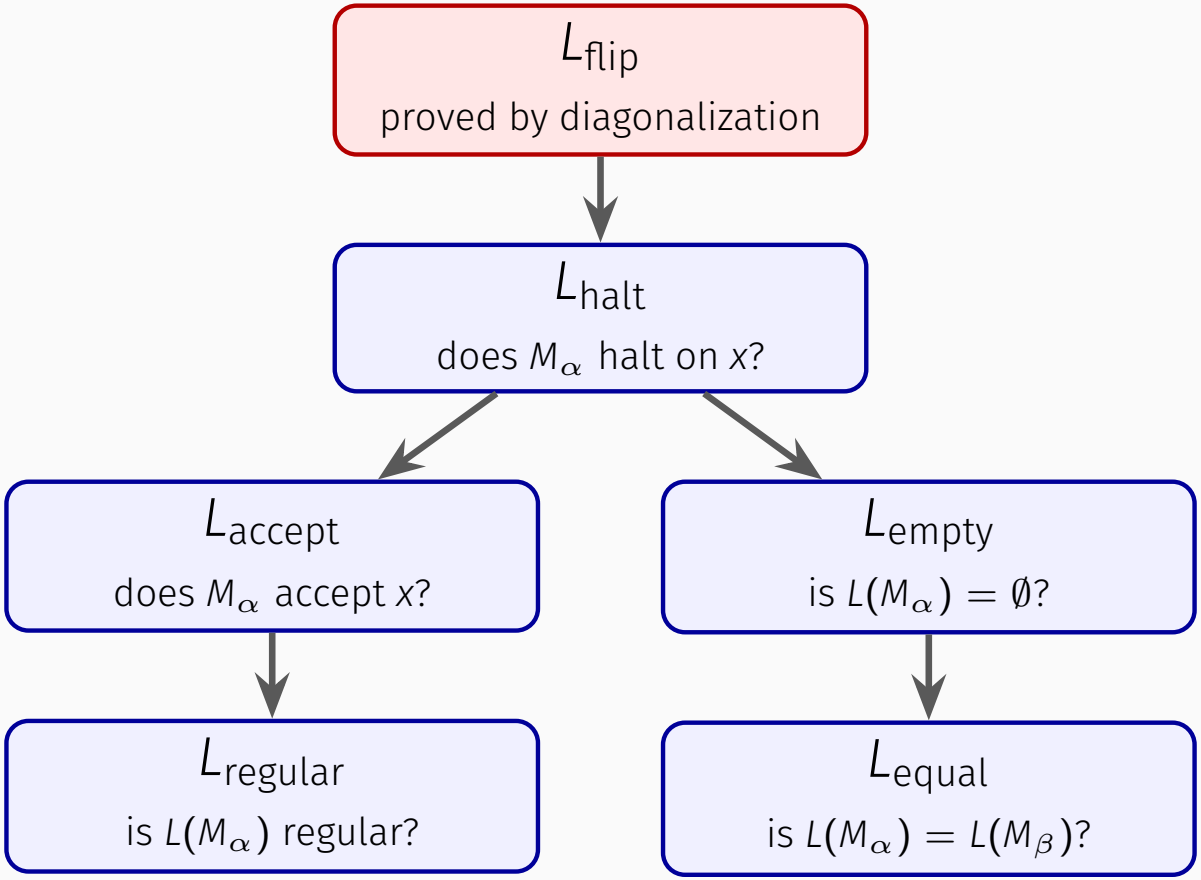
Proof.
Suppose M_{equal} decides L_{equal}. We decide L_{empty} as follows.

Let M_γ reject every input, so $L(M_\gamma) = \emptyset$. Given α, run M_{equal} on (α, γ). Since

$$L(M_\alpha) = \emptyset \iff L(M_\alpha) = L(M_\gamma),$$

this decides L_{empty}. Contradiction. $\square$

The Reduction Hierarchy



$A \longrightarrow B$: if B were decidable, then A would be decidable too.

Exercise: Decidable or Undecidable?

Is each language below decidable? Justify your answer.

1. $L_\varepsilon = \{\alpha : M_\alpha \text{ accepts the empty string } \varepsilon\}$
2. $L_{\leq 10} = \{\alpha : M_\alpha \text{ has at most 10 states}\}$
3. $L_{\text{all}} = \{\alpha : L(M_\alpha) = \Sigma^*\}$
4. $L_{100} = \{(\alpha, x) : M_\alpha \text{ halts on } x \text{ within 100 steps}\}$
5. $L_{\text{inf}} = \{\alpha : L(M_\alpha) \text{ is infinite}\}$

Nontrivial Properties of Languages

Definition 15
Let $P \subseteq \{0,1\}^*$ be a set of TM encodings.

P is a *property of languages* if for all $\alpha, \beta \in \{0,1\}^*$,

$$L(M_\alpha) = L(M_\beta) \implies (\alpha \in P \iff \beta \in P).$$

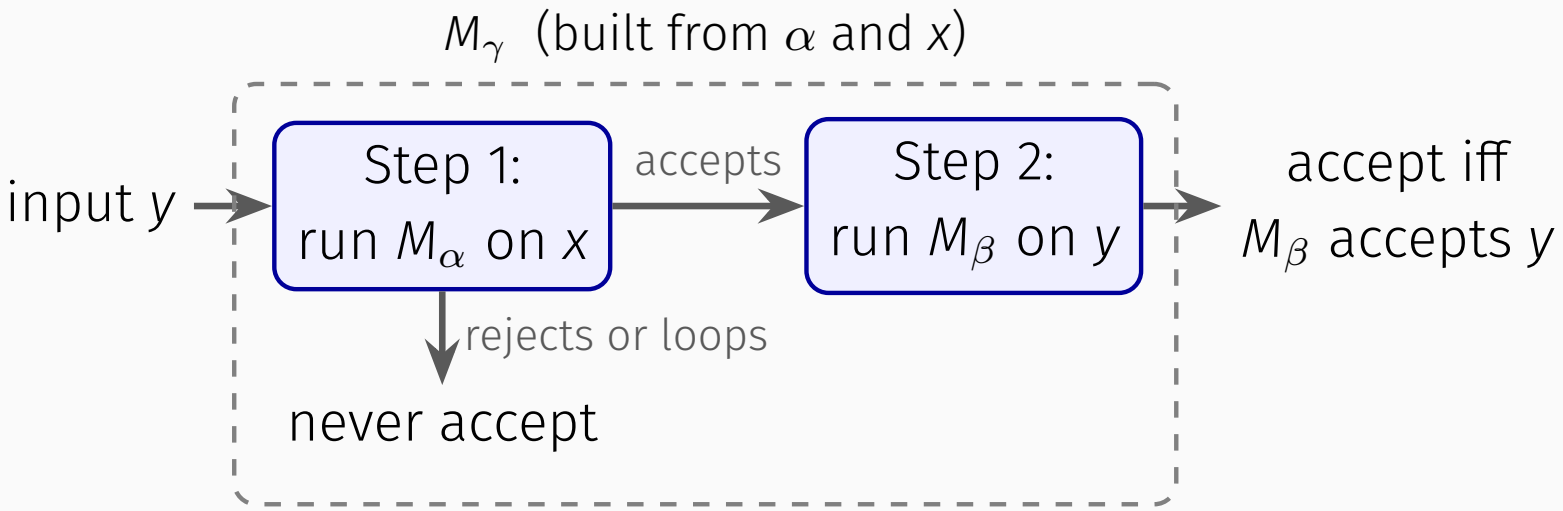
P is *nontrivial* if $P \neq \emptyset$ and $P \neq \{0,1\}^*$.

Rice's Theorem

Theorem 16 (Rice, 1953)
If P is a nontrivial property of languages, then P is undecidable.

Consequence: $\{\alpha : L(M_\alpha) \text{ is } \ldots\}$ is undecidable, where "..." can be regular, empty, finite, contains 0100, has an odd-length string, ...

Proof Idea of Rice's Theorem



M_α accepts x

$\Rightarrow \quad L(M_\gamma) = L(M_\beta)$

$\Rightarrow \quad \gamma \in P$

M_α does not accept x

$\Rightarrow \quad L(M_\gamma) = \emptyset = L(M_\delta)$

$\Rightarrow \quad \gamma \notin P$

A decider for P tells the two cases apart $\Rightarrow$ it decides L_{accept}.

Proof of Rice's Theorem

Fix δ with $L(M_\delta) = \emptyset$. WLOG $\delta \notin P$ (else replace P by its complement). Since $P \neq \emptyset$, fix some $\beta \in P$.

Given (α, x), build M_γ: on input y, first simulate M_α on x; if it accepts, simulate M_β on y and accept iff M_β accepts.

$$L(M_\gamma) = \begin{cases} L(M_\beta), \text{ so } \gamma \in P & \text{if } M_\alpha \text{ accepts } x, \\ \emptyset = L(M_\delta), \text{ so } \gamma \notin P & \text{otherwise.} \end{cases}$$

So $(\alpha, x) \in L_{\text{accept}} \iff \gamma \in P$. A decider for P would decide L_{accept}. $\square$